

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Разработка мобильных приложений

Отчет

Курсовая работа

Выполнил:

Дмитриев Андрей

Группа

K3439

Санкт-Петербург

2026 г.

1. ВВЕДЕНИЕ

1.1 Цель работы

Цель производственной практики — разработать полнофункциональное мобильное приложение «Мессенджер» для Android с применением современных компонентов Jetpack и архитектурных подходов.

1.2 Задачи

1. Изучить жизненный цикл Activity и Fragment.
2. Реализовать навигацию между экранами через Navigation Component.
3. Применить архитектурный паттерн MVVM.
4. Настроить сетевые запросы через Retrofit и локальную БД через Room.
5. Реализовать фоновую синхронизацию через WorkManager.
6. Реализовать уведомления и обработку разрешений.
7. Сделать современный UI с компонентами Material Design.

1.3 Актуальность

Мессенджеры — один из самых востребованных типов мобильных приложений. Разработка такого приложения позволяет закрепить основные навыки Android-разработки: UI, навигация, архитектура, работа с сетью и локальным хранилищем, фоновая синхронизация и уведомления.

2. ОПИСАНИЕ ПРИЛОЖЕНИЯ

2.1 Общее описание

Приложение «Мессенджер» предназначено для просмотра ленты сообщений и взаимодействия с ней. Данные загружаются из сети, сохраняются локально в базе данных и доступны в офлайн-режиме. Реализована возможность ставить лайки сообщениям.

2.2 Основные функции (схема вместо таблицы)

СХЕМА: ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ

- Лента сообщений: список сообщений с аватарами пользователей

- Лайки: отметка понравившихся сообщений и сохранение состояния
- Профиль пользователя: редактирование имени и статуса
- Настройки: переключение светлой/тёмной темы
- Фоновая синхронизация: периодическое обновление данных
- Уведомления: информирование о завершении синхронизации/новых данных
- Офлайн-режим: работа с кэшем при отсутствии интернета

2.3 Экраны приложения (схема)



СХЕМА: ОСНОВНЫЕ ЭКРАНЫ

- Экран “Новости / Лента”: список сообщений + кнопка обновления (FAB)
- Экран “Профиль”: поля “Имя” и “Статус”, сохранение изменений
- Экран “Настройки”: переключатель темы, отображение версии приложения
- Нижняя навигация: [Новости] [Профиль] [Настройки]

3. АРХИТЕКТУРА ПРИЛОЖЕНИЯ

3.1 Архитектурный паттерн MVVM

Приложение реализовано по паттерну MVVM (Model–View–ViewModel). View (Fragment) наблюдает за состоянием в ViewModel (LiveData/Flow). ViewModel обращается к Repository, который выбирает источник данных (Room или Retrofit) и отвечает за кэширование/синхронизацию.

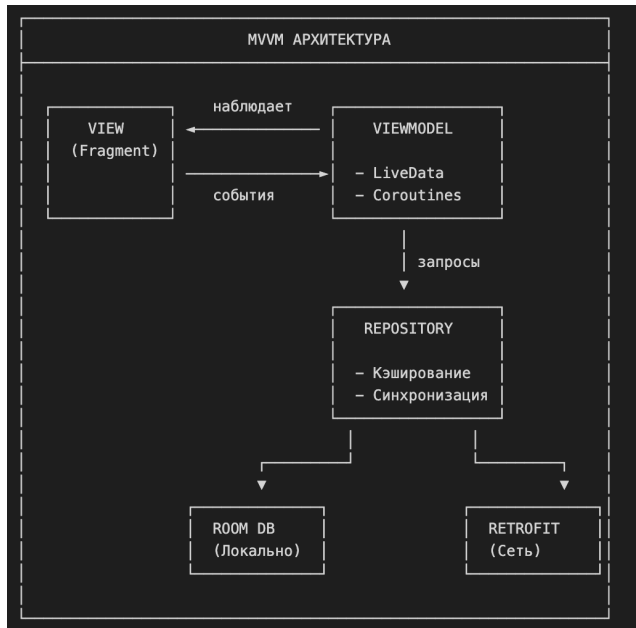


СХЕМА: MVVM

VIEW (Fragment/UI)

↓ observe / render

VIEWMODEL (состояние UI, события, корутины)

↓ запросы данных

REPOSITORY (единая точка доступа к данным, кэш, синхронизация)

↓ ↓

ROOM (локально) RETROFIT (сеть)

3.2 Структура проекта (схема)

СХЕМА: СТРУКТУРА ПРОЕКТА

app/

- adapter/
 - MessageAdapter.kt — RecyclerView адаптер
- api/
 - MessageApiService.kt — интерфейс Retrofit
 - RetrofitClient.kt — настройка клиента
- data/
 - AppDatabase.kt — база данных Room
 - Message.kt — модель/Entity

- MessageDao.kt — DAO
- fragments/
 - NewsFeedFragment.kt — лента
 - ProfileFragment.kt — профиль
 - SettingsFragment.kt — настройки
- repository/
 - MessageRepository.kt — репозиторий данных
- utils/
 - NetworkMonitor.kt — мониторинг сети
 - NotificationHelper.kt — уведомления
- viewmodel/
 - MessageViewModel.kt — VM ленты
 - ProfileViewModel.kt — VM профиля
 - SettingsViewModel.kt — VM настроек
- worker/
 - SyncWorker.kt — фоновая синхронизация
- MainActivity.kt — главная Activity
- MessengerApplication.kt — Application класс

3.3 Диаграмма классов (упрощённая текстовая схема)

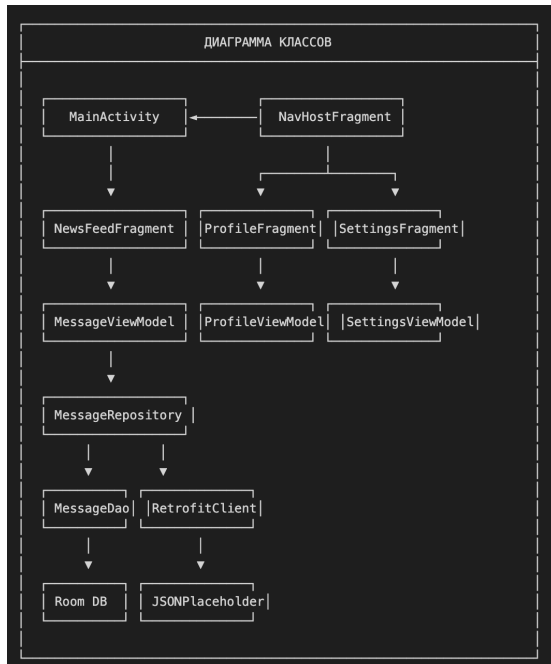


СХЕМА: ВЗАИМОСВЯЗИ КОМПОНЕНТОВ

MainActivity

- └ NavHostFragment
 - └ NewsFeedFragment → MessageViewModel → MessageRepository
 - └ ProfileFragment → ProfileViewModel
 - └ SettingsFragment → SettingsViewModel

MessageRepository

- └ MessageDao → Room DB
- └ RetrofitClient → MessageApiService → API

4. ЛАБОРАТОРНАЯ РАБОТА №1 — ОСНОВЫ ANDROID

4.1 Цель работы

Изучить основы Android-разработки, жизненный цикл Activity и подготовить базовый UI-проект.

4.2 Жизненный цикл Activity (схема)

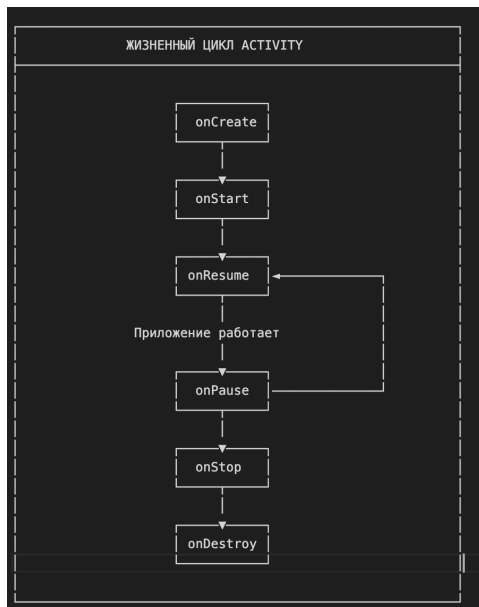


СХЕМА: ЖИЗНЕННЫЙ ЦИКЛ ACTIVITY

onCreate() → onStart() → onResume()

(приложение активно)

onPause() → onStop() → onDestroy()

При возвращении в приложение обычно вызываются onRestart() → onStart() → onResume()

4.3 Листинг: MainActivity.kt

(код оставлен без изменений)

```
package com.example.messenger
```

```
import android.os.Bundle
```

```
import android.util.Log
```

```
import androidx.appcompat.app.AppCompatActivity
```

```
import com.example.messenger.databinding.ActivityMainBinding
```

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var binding: ActivityMainBinding
```

```
    private val TAG = "MainActivity"
```

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    Log.d(TAG, "onCreate: MainActivity создана")  
  
    binding = ActivityMainBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
}
```

```
override fun onStart() {  
    super.onStart()  
    Log.d(TAG, "onStart: MainActivity стартовала")  
}
```

```
override fun onResume() {  
    super.onResume()  
    Log.d(TAG, "onResume: MainActivity возобновлена")  
}
```

```
override fun onPause() {  
    super.onPause()  
    Log.d(TAG, "onPause: MainActivity приостановлена")  
}
```

```
override fun onStop() {  
    super.onStop()
```



```

        Log.d(TAG, "onStop: MainActivity остановлена")
    }

    override fun onDestroy() {
        super.onDestroy()

        Log.d(TAG, "onDestroy: MainActivity уничтожена")
    }
}

```

4.4 Результаты

- Создан базовый проект Android-приложения.
- Изучен жизненный цикл Activity.
- Реализовано логирование основных состояний.
- Подключён ViewBinding для работы с UI.

5. ЛАБОРАТОРНАЯ РАБОТА №2 — НАВИГАЦИЯ И MVVM

5.1 Цель работы

Сделать навигацию между экранами через Navigation Component и применить MVVM.

5.2 Навигация между фрагментами (схема)

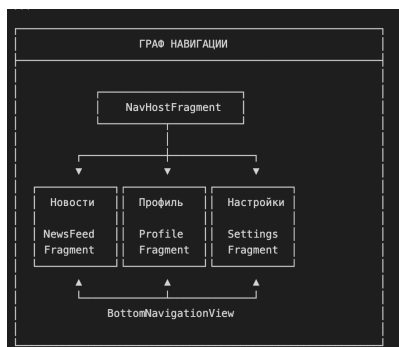


СХЕМА: НАВИГАЦИЯ

NavHostFragment (контейнер)

└─ NewsFeedFragment

└─ ProfileFragment

└─ SettingsFragment

Переключение: BottomNavigationView

5.3 Листинг: nav_graph.xml

(код оставлен без изменений)

```
<?xml version="1.0" encoding="utf-8"?>

<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/navigation_news">

    <fragment
        android:id="@+id/navigation_news"
        android:name="com.example.messenger.fragments.NewsFeedFragment"
        android:label="@string/title_news" />

    <fragment
        android:id="@+id/navigation_profile"
        android:name="com.example.messenger.fragments.ProfileFragment"
        android:label="@string/title_profile" />

    <fragment
        android:id="@+id/navigation_settings"
        android:name="com.example.messenger.fragments.SettingsFragment"
        android:label="@string/title_settings" />

</navigation>
```

5.4 Листинг: ProfileViewModel.kt

(код оставлен без изменений)

```
package com.example.messenger.viewmodel

import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class ProfileViewModel : ViewModel() {

    private val _userName = MutableLiveData<String>()
    val userName: LiveData<String> = _userName

    private val _userStatus = MutableLiveData<String>()
    val userStatus: LiveData<String> = _userStatus

    init {
        _userName.value = "Иван Иванов"
        _userStatus.value = "Android разработчик"
    }

    fun updateUserName(name: String) {
        _userName.value = name
    }

    fun updateUserStatus(status: String) {
```

```

        _userStatus.value = status
    }
}

```

5.5 Результаты

- Настроен Navigation Component и граф навигации.
- Добавлен BottomNavigationView для переключения экранов.
- Применён MVVM: ViewModel + LiveData.
- Данные ViewModel сохраняются при повороте экрана.

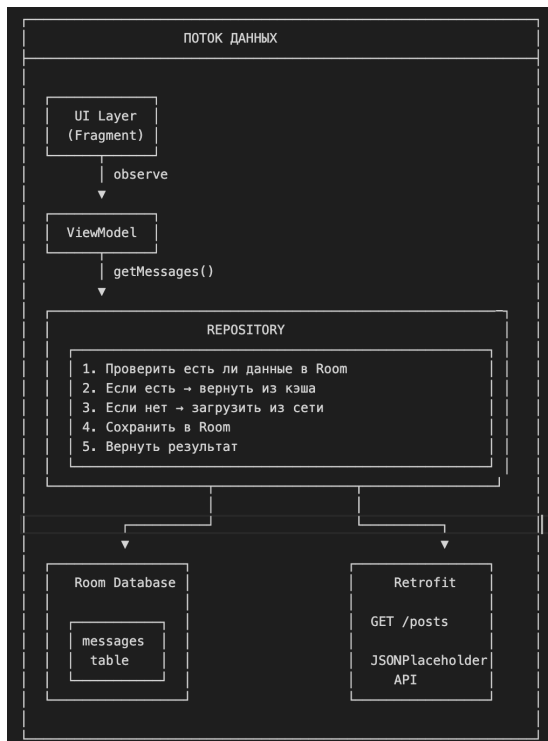
6. ЛАБОРАТОРНАЯ РАБОТА №3 — РАБОТА С СЕТЬЮ И БАЗОЙ ДАННЫХ

6.1 Цель работы

Реализовать загрузку данных из сети через Retrofit и кэширование в Room.

6.2 Схема работы с данными (схема)

СХЕМА: ПОТОК ДАННЫХ



UI (Fragment)

↓ observe

ViewModel

↓ запрос данных

Repository

1. проверить кэш в Room
2. если есть данные → вернуть из БД
3. иначе → запрос в сеть (Retrofit)
4. сохранить результат в Room
5. вернуть результат в UI

6.3 Листинг: Message.kt (Entity)

```
package com.example.messenger.data
```

```
import androidx.room.Entity
```

```
import androidx.room.PrimaryKey
```

```
@Entity(tableName = "messages")
```

```
data class Message(
```

```
    @PrimaryKey val id: Int,
```

```
    val userId: Int,
```

```
    val title: String,
```

```
    val body: String,
```

```
    val isLiked: Boolean = false,
```

```
    val avatarColor: Int = 0
```

```
)
```

6.4 Листинг: MessageDao.kt

```
package com.example.messenger.data
```

```
import androidx.room.Dao
```

```

import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

@Dao
interface MessageDao {

    @Query("SELECT * FROM messages ORDER BY id DESC")
    fun getAllMessagesFlow(): Flow<List<Message>>

    @Query("SELECT * FROM messages ORDER BY id DESC")
    suspend fun getAllMessages(): List<Message>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMessages(messages: List<Message>)

    @Query("UPDATE messages SET isLiked = :isLiked WHERE id = :messageId")
    suspend fun updateLikeStatus(messageId: Int, isLiked: Boolean)

    @Query("DELETE FROM messages")
    suspend fun deleteAllMessages()
}

```

6.5 Листинг: MessageApiService.kt

```

package com.example.messenger.api

```

```
import retrofit2.http.GET

interface MessageApiService {

    @GET("posts")
    suspend fun getMessages(): List<MessageResponse>

}
```

```
data class MessageResponse(
    val id: Int,
    val userId: Int,
    val title: String,
    val body: String
){
    fun toMessage(): Message {
        return Message(
            id = id,
            userId = userId,
            title = title,
            body = body,
            isLiked = false,
            avatarColor = generateAvatarColor(userId)
        )
    }
}
```

6.6 Листинг: MessageRepository.kt

```
package com.example.messenger.repository
```

```
import com.example.messenger.api.RetrofitClient
import com.example.messenger.data.Message
import com.example.messenger.data.MessageDao
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext

class MessageRepository(private val messageDao: MessageDao) {

    suspend fun getMessages(forceRefresh: Boolean = false): Result<List<Message>> {
        return withContext(Dispatchers.IO) {
            try {
                if (forceRefresh) {
                    fetchFromApi()
                } else {
                    val cachedMessages = messageDao.getAllMessages()
                    if (cachedMessages.isNotEmpty()) {
                        Result.success(cachedMessages)
                    } else {
                        fetchFromApi()
                    }
                }
            }
        }
    }

    } catch (e: Exception) {
        val cachedMessages = messageDao.getAllMessages()
        if (cachedMessages.isNotEmpty()) {
            Result.success(cachedMessages)
        }
    }
}
```



```

        } else {
            Result.failure(e)
        }
    }
}
}
}
}

```

```

private suspend fun fetchFromApi(): Result<List<Message>> {
    val responses = RetrofitClient.messageApi.getMessages()
    val messages = responses.map { it.toMessage() }
    messageDao.deleteAllMessages()
    messageDao.insertMessages(messages)
    return Result.success(messages)
}
}

```

6.7 Результаты

- Настроен Retrofit для REST API.
- Реализована локальная база Room для кэширования.
- Применён Repository-подход.
- Реализована работа в офлайн-режиме при наличии кэша.

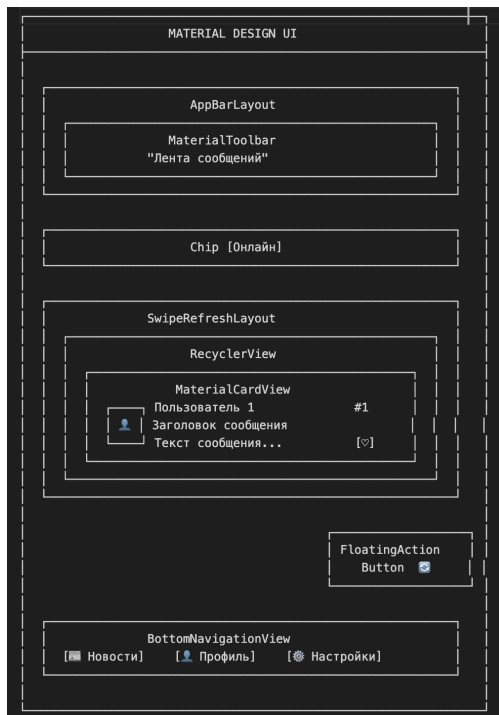
7. ЛАБОРАТОРНАЯ РАБОТА №4 — UI, УВЕДОМЛЕНИЯ И ФОНОВАЯ СИНХРОНИЗАЦИЯ

7.1 Цель работы

Улучшить UI с Material Design, добавить уведомления и фоновую синхронизацию через WorkManager.

7.2 Material Design компоненты (схема)

СХЕМА: UI НА MATERIAL



- AppBarLayout + MaterialToolbar (заголовок экрана)
- RecyclerView (лента сообщений)
- MaterialCardView (карточка сообщения)
- SwipeRefreshLayout (обновление свайпом)
- FloatingActionButton (обновление/действие)
- BottomNavigationView (переключение экранов)
- Chip (например, "Онлайн"/фильтр)

7.3 Листинг: MessageAdapter.kt

```
package com.example.messenger.adapter
```

```
import android.graphics.drawable.GradientDrawable
```

```
import android.view.LayoutInflater
```

```
import android.view.ViewGroup
```

```
import androidx.recyclerview.widget.DiffUtil
```

```
import androidx.recyclerview.widget.ListAdapter
```

```
import androidx.recyclerview.widget.RecyclerView
import com.example.messenger.R
import com.example.messenger.data.Message
import com.example.messenger.databinding.ItemMessageBinding

class MessageAdapter(
    private val onLikeClick: (Message) -> Unit
) : ListAdapter<Message, MessageAdapter.MessageViewHolder>(MessageDiffCallback()) {

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MessageViewHolder {
        {
            val binding = ItemMessageBinding.inflate(
                LayoutInflater.from(parent.context),
                parent,
                false
            )
            return MessageViewHolder(binding, onLikeClick)
        }
    }

    override fun onBindViewHolder(holder: MessageViewHolder, position: Int) {
        holder.bind(getItem(position))
    }
}

class MessageViewHolder(
    private val binding: ItemMessageBinding,
    private val onLikeClick: (Message) -> Unit
```

```
) : RecyclerView.ViewHolder(binding.root) {

fun bind(message: Message) {
    binding.apply {
        textUserName.text = "Пользователь ${message.userId}"
        textMessageTitle.text = message.title
        textMessageBody.text = message.body
        textMessageId.text = "#${message.id}"

        val avatarDrawable = GradientDrawable().apply {
            shape = GradientDrawable.OVAL
            setColor(message.avatarColor)
        }
        imageAvatar.background = avatarDrawable

        val iconRes = if (message.isLiked) {
            R.drawable.ic_favorite_filled
        } else {
            R.drawable.ic_favorite_border
        }
        buttonLike.setImageResource(iconRes)

        buttonLike.setOnClickListener {
            onLikeClick(message)
        }
    }
}
```

```

    }
}

private class MessageDiffCallback : DiffUtil.ItemCallback<Message>() {

    override fun areItemsTheSame(oldItem: Message, newItem: Message) =

        oldItem.id == newItem.id

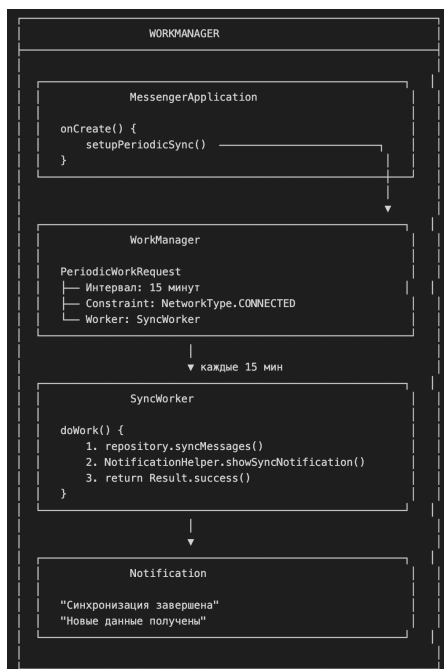
    override fun areContentsTheSame(oldItem: Message, newItem: Message) =

        oldItem == newItem

}
}

```

7.4 WorkManager — фоновая синхронизация (схема)



7.5 Листинг: SyncWorker.kt

```
package com.example.messenger.worker
```

```
import android.content.Context
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import com.example.messenger.data.AppDatabase
import com.example.messenger.repository.MessageRepository
import com.example.messenger.utils.NotificationHelper

class SyncWorker(
    context: Context,
    workerParams: WorkerParameters
) : CoroutineWorker(context, workerParams) {

    override suspend fun doWork(): Result {
        return try {
            val database = AppDatabase.getDatabase(applicationContext)
            val repository = MessageRepository(database.messageDao())

            val syncResult = repository.syncMessages()

            if (syncResult) {
                NotificationHelper.showSyncNotification(applicationContext)
                Result.success()
            } else {
                Result.retry()
            }
        } catch (e: Exception) {
```

```

        Result.failure()
    }
}

companion object {
    const val WORK_NAME = "sync_messages_work"
}
}

```

7.6 Листинг: NotificationHelper.kt

```

package com.example.messenger.utils

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.os.Build
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat
import com.example.messenger.MainActivity
import com.example.messenger.R

object NotificationHelper {

    private const val CHANNEL_ID = "sync_channel"
    private const val NOTIFICATION_ID = 1001

```

```

fun createNotificationChannel(context: Context) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            CHANNEL_ID,
            "Синхронизация",
            NotificationManager.IMPORTANCE_DEFAULT
        )
        val manager = context.getSystemService(NotificationManager::class.java)
        manager.createNotificationChannel(channel)
    }
}

```

```

fun showSyncNotification(context: Context) {
    val intent = Intent(context, MainActivity::class.java)
    val pendingIntent = PendingIntent.getActivity(
        context, 0, intent,
        PendingIntent.FLAG_IMMUTABLE
    )
}

```

```

val notification = NotificationCompat.Builder(context, CHANNEL_ID)
    .setSmallIcon(R.drawable.ic_sync)
    .setContentTitle("Синхронизация завершена")
    .setContentText("Новые данные получены")
    .setContentIntent(pendingIntent)
    .setAutoCancel(true)

```



```
.build()
```

```
NotificationManagerCompat.from(context).notify(NOTIFICATION_ID, notification)
```

```
}
```

```
}
```

7.7 Мониторинг сети

```
package com.example.messenger.utils
```

```
import android.content.Context
```

```
import android.net.ConnectivityManager
```

```
import android.net.Network
```

```
import android.net.NetworkCapabilities
```

```
import android.net.NetworkRequest
```

```
import kotlinx.coroutines.channels.awaitClose
```

```
import kotlinx.coroutines.flow.Flow
```

```
import kotlinx.coroutines.flow.callbackFlow
```

```
class NetworkMonitor(private val context: Context) {
```

```
    private val connectivityManager =
```

```
        context.getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
```

```
    fun isConnected(): Boolean {
```

```
        val network = connectivityManager.activeNetwork
```

```
        val capabilities = connectivityManager.getNetworkCapabilities(network)
```

```
        return capabilities?.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
    == true
}
```

```
val networkState: Flow<NetworkState> = callbackFlow {
    val callback = object : ConnectivityManager.NetworkCallback() {
        override fun onAvailable(network: Network) {
            trySend(NetworkState.Connected)
        }
        override fun onLost(network: Network) {
            trySend(NetworkState.Disconnected)
        }
    }
}
```

```
val request = NetworkRequest.Builder()
    .addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
    .build()
```

```
connectivityManager.registerNetworkCallback(request, callback)
```

```
awaitClose {
    connectivityManager.unregisterNetworkCallback(callback)
}
}
```

```
sealed class NetworkState {
```

```

        object Connected : NetworkState()
        object Disconnected : NetworkState()
    }
}

```

7.8 Запрос разрешений (пример)

```

private val requestPermissionLauncher = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
){ permissions ->
    permissions.entries.forEach { entry ->
        Log.d(TAG, "${entry.key}: ${entry.value}")
    }
}

private fun requestPermissions() {
    val permissionsToRequest = mutableListOf<String>()

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS)
            != PackageManager.PERMISSION_GRANTED) {
            permissionsToRequest.add(Manifest.permission.POST_NOTIFICATIONS)
        }
    }

    if (permissionsToRequest.isNotEmpty()) {
        requestPermissionLauncher.launch(permissionsToRequest.toTypedArray())
    }
}

```

}

7.9 Результаты

- UI приведён к современному виду с Material Design (карточки, FAB, toolbar, нижняя навигация).
- Добавлены лайки и сохранение состояния.
- Настроена фоновая синхронизация через WorkManager.
- Добавлены уведомления о синхронизации.
- Реализован мониторинг сети.
- Реализован запрос разрешений (в т.ч. уведомления Android 13+).

8. ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

8.1 Основные технологии (схема)

СХЕМА: ОСНОВА

- Kotlin 1.9.20 — основной язык
- Android SDK 34 — платформа
- Gradle 8.2 — сборка проекта

8.2 Android Jetpack (схема)

СХЕМА: JETPACK

- Navigation 2.7.6 — переходы между экранами
- Room 2.6.1 — локальная БД
- ViewModel 2.7.0 — состояние UI
- LiveData 2.7.0 — наблюдение за данными
- WorkManager 2.9.0 — фоновые задачи
- ViewBinding — удобная работа с UI

8.3 Сторонние библиотеки (схема)

СХЕМА: БИБЛИОТЕКИ

- Retrofit 2.9.0 — сетевые запросы

- OkHttp 4.12.0 — HTTP-клиент
- Gson 2.10.1 — парсинг JSON
- Coroutines 1.7.3 — асинхронность
- Material 1.11.0 — UI-компоненты

8.4 Зависимости (build.gradle.kts)

```
dependencies {
```

```
    implementation("androidx.core:core-ktx:1.12.0")
```

```
    implementation("androidx.appcompat:appcompat:1.6.1")
```

```
    implementation("com.google.android.material:material:1.11.0")
```

```
    implementation("androidx.navigation:navigation-fragment-ktx:2.7.6")
```

```
    implementation("androidx.navigation:navigation-ui-ktx:2.7.6")
```

```
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
```

```
    implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")
```

```
    implementation("androidx.room:room-runtime:2.6.1")
```

```
    implementation("androidx.room:room-ktx:2.6.1")
```

```
    ksp("androidx.room:room-compiler:2.6.1")
```

```
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
```

```
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")
```

```
    implementation("androidx.work:work-runtime-ktx:2.9.0")
```

```
    implementation("org.jetbrains.kotlin:kotlin-coroutines-android:1.7.3")
```

}

9. ЗАКЛЮЧЕНИЕ

9.1 Результаты работы

В ходе практики разработано Android-приложение «Мессенджер», включающее:

1. Ленту сообщений (загрузка из сети и отображение в RecyclerView).
2. Лайки (изменение состояния и хранение).
3. Профиль пользователя (редактирование имени и статуса).
4. Настройки (переключение темы).
5. Офлайн-режим (работа с кэшем в Room).
6. Фоновую синхронизацию (WorkManager).
7. Уведомления (о синхронизации и обновлениях).

9.2 Полученные навыки

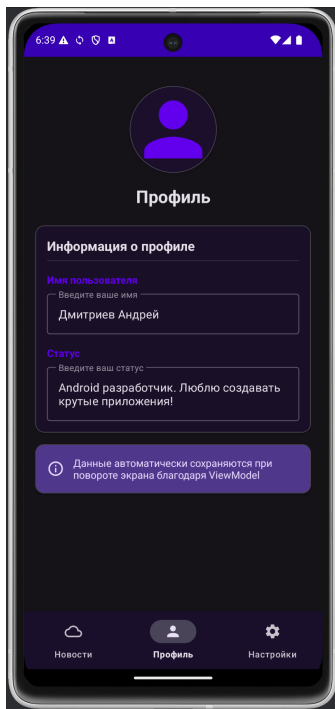
- Жизненный цикл Activity/Fragment.
- MVVM, ViewModel, LiveData/Flow.
- Navigation Component и граф навигации.
- Retrofit + корутины.
- Room и DAO.
- WorkManager и фоновые задачи.
- Уведомления и разрешения.
- Material Design UI.

9.3 Скриншоты приложения

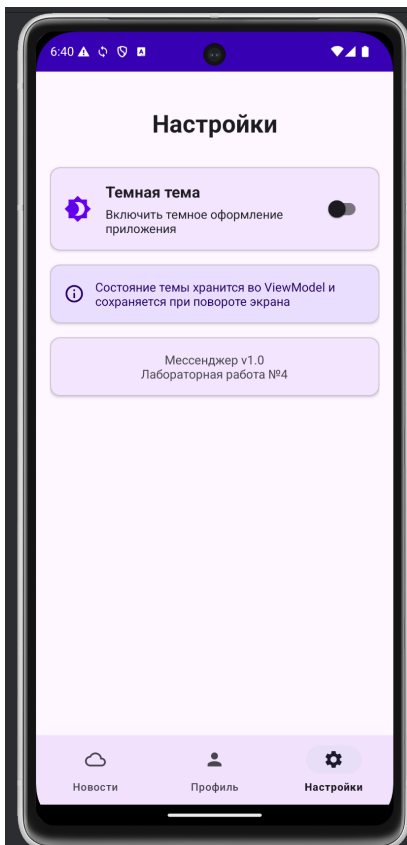
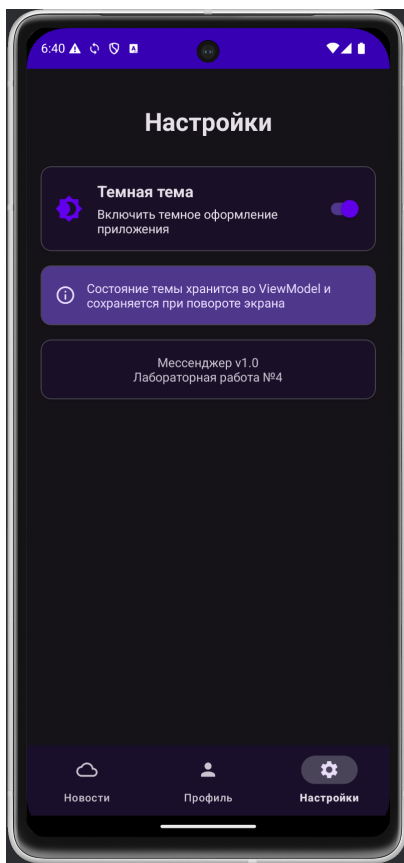
- Скрин 1: Экран “Лента”



- Скрин 2: Экран “Профиль”



- Скрин 3: Экран “Настройки”



9.4 Перспективы развития

- Авторизация пользователей.
- Реальная отправка сообщений (чат).
- Push-уведомления через Firebase Cloud Messaging.
- Подключение к реальному backend-серверу.
- Шифрование сообщений.